

PROJECT FOR PRODUCT DEVELOPMENT REPORT

on

LOAD BALANCING IN SDN

**Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in Computer Science & Engineering**

6th Semester

Submitted by

Akash Behera	2301109366
Sidhanta Kumar Nayak	2421109115
Ritesh Sahu	2201109076
Jiten Patnaik	2421109111
Sayed Naquib Iqbal	2421109114

Under the guidance of

Dr. Mitali Sinha
Mrs. Chapala Maharana
Assistant Professor
Department of CSE

**Department of Computer Science & Engineering
Parala Maharaja Engineering College, Berhampur, Odisha**

Academic Session: 2025-26

Certificate

This is to certify that the report entitled “*Load Balancing in SDN*” submitted by **Akash Behera, Sidhant Nayak, Jiten Patnaik, Ritesh Sahu, Sayed Nauib Iqbal** bearing Registration No(s) **2301109366,2421109115,2421109111,2201109076 and 2421109114** is a bonafide record of work carried out by them under my supervision and guidance in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering** of Biju Patnaik University of Technology, Odisha, during the academic session **2025–2026**.

Signature of Mentors

Signature of HOD

Date:

Place:

Declaration

We hereby declare that the report entitled “*Load Balancing in SDN*” is an authentic record of our own work carried out under the supervision of **Dr. Mitali Sinha**. The work has not been submitted elsewhere for the award of any degree or diploma.

Signature of Students

Student Name(s) :

Akash Behera

Sidhanta Kumar Nayak

Ritesh Sahu

Jiten Patnaik

Sayed Naquib Iqbal

Registration No(s):

2301109366

2421109115

2201109076

2421109111

2421109114

Date:

Place:

Acknowledgement

We express our sincere gratitude to **Dr. Mitali Sinha and Mrs. Chapala Maharana** for their valuable guidance, encouragement, and support throughout the course of this project.

We also thank the **Department of Computer Science & Engineering of Parala Maharaja Engineering College** for providing necessary facilities to carry out this work.

We extend our thanks to all those who directly or indirectly contributed to the successful completion of this project.

Abstract

Software Defined Networking (SDN) decouples the network control plane from the data plane, enabling centralized and programmable network management. One of the most critical challenges in SDN environments is efficient load balancing across network links and switches to prevent congestion and ensure Quality of Service (QoS). This project proposes and implements a weighted load balancing algorithm for SDN that considers three key network metrics: available bandwidth, link latency, and packet loss rate. Each metric is assigned a configurable weight, and traffic flows are distributed to the optimal path using a composite score computed from these weighted parameters. The solution is implemented using the Ryu SDN controller and evaluated on a Mininet network emulation environment. Experimental results demonstrate that the proposed weighted approach achieves significantly better throughput, lower average latency, and reduced packet loss compared to traditional round-robin and random load balancing methods. The system proves effective in dynamically adapting to changing network conditions in real time.

Keywords: Software Defined Networking, Load Balancing, Weighted Metrics, Bandwidth, Latency, Packet Loss, Ryu Controller, Mininet, OpenFlow.

Contents

1	Introduction	7
1.1	Background	7
1.2	Problem Statement	8
1.3	Objectives	9
1.4	Requirements Analysis	10
2	Literature Review & Existing Digital Solutions	12
2.1	Summary of Survey	12
2.2	Identified Gaps	13
3	System Design	14
3.1	High-level Architecture	14
3.2	Module Description	15
3.3	UML Diagrams	16
3.3.1	Use Case Diagram	16
3.3.2	Class Diagram	17
3.3.3	Activity Diagram	18
3.3.4	Sequence Diagram	19
3.4	Algorithms/Methods Used	20
4	Implementation	22
4.1	Technology Stack	22
4.2	Key Features Implemented	23
5	Testing and Validation	25
5.1	Test Cases	25
5.2	Performance Analysis	26
5.3	User Feedback	27
6	Results and Discussion	28

7	Deployment	29
7.1	Build Process	29
7.2	Installation Steps	30
7.3	Deployment Platform	31
8	Conclusion and Future Work	32
8.1	Conclusion	32
8.2	Future Enhancements	33
A	Appendix	35
A.1	Source Code (Optional)	35
A.2	Additional Screenshots	35

Chapter 1

Introduction

1.1 Background

In traditional networking systems, the control and data forwarding functions are tightly coupled within network devices such as routers and switches. Each device independently makes forwarding decisions, which results in a distributed control mechanism. This approach leads to several limitations, including complex network management, lack of flexibility, and difficulty in adapting to changing network conditions, especially in large-scale environments.

With the rapid growth of internet usage, cloud computing, and data-intensive applications, network traffic has increased significantly. This increase in traffic often leads to congestion, higher latency, packet loss, and inefficient utilization of available network resources. As a result, there is a need for a more flexible and efficient network management system.

Software Defined Networking (SDN) has emerged as a promising solution to address these challenges. SDN separates the control plane from the data plane, allowing centralized control of the network through a controller. This centralized approach simplifies network management and enables dynamic configuration of network resources.

One of the key issues in SDN is load balancing, which is essential for distributing network traffic efficiently across multiple available paths. Traditional load balancing techniques such as Round Robin and Least Connection do not consider real-time network conditions. These methods often lead to uneven traffic distribution and poor performance, as they ignore important parameters like bandwidth, latency, and packet loss.

To overcome these limitations, a weighted load balancing approach can be used, where multiple network parameters are considered for decision-making. By assigning weights based on bandwidth, latency, and packet loss, the system can dynamically select the most efficient path for data transmission. This approach helps in reducing congestion, improving throughput, and enhancing overall network performance.

1.2 Problem Statement

In modern networking environments, the rapid increase in data traffic due to cloud services, multimedia applications, and large-scale communication systems has led to significant challenges in managing network resources efficiently. Traditional networking architectures rely on distributed control mechanisms, where each device independently makes routing decisions. This often results in inefficient traffic distribution, network congestion, increased latency, and higher packet loss.

Even in Software Defined Networking (SDN), which offers centralized control and programmability, efficient load balancing remains a critical challenge. Existing load balancing techniques such as Round Robin and Least Connection do not consider real-time network conditions and fail to account for important performance parameters like bandwidth availability, latency, and packet loss. As a result, traffic may be routed through suboptimal paths, leading to poor network performance and underutilization of available resources.

Therefore, there is a need for an intelligent load balancing mechanism that can dynamically adapt to changing network conditions. The problem addressed in this project is to design and implement a weighted load balancing algorithm in SDN that considers multiple parameters—bandwidth, latency, and packet loss—to select the most efficient path for data transmission. The objective is to improve network performance by reducing congestion, minimizing delay and packet loss, and ensuring optimal utilization of network resources.

1.3 Objectives

The main objective of this project is to design and implement an efficient load balancing mechanism in Software Defined Networking (SDN) using a weighted algorithm. The proposed system aims to improve overall network performance by considering multiple important parameters such as bandwidth, latency, and packet loss. By dynamically selecting the optimal path for data transmission, the system ensures better utilization of network resources and enhanced Quality of Service (QoS).

Specific Objectives;

To design a weighted load balancing algorithm based on multiple network parameters.

To consider key performance metrics such as:

Bandwidth

Latency

Packet Loss

To dynamically select the best path for data transmission in the network.

To reduce network congestion by efficient traffic distribution.

To improve throughput and bandwidth utilization.

To minimize latency and packet loss during data transmission.

To implement the proposed system using SDN tools and simulation environment.

To compare the performance of the proposed method with traditional load balancing technique

1.4 Requirements Analysis

Requirement analysis is an important phase in system development that involves identifying and specifying the functional and non-functional requirements of the system. ensures that the proposed system meets user needs and performs efficiently under different network conditions.

1.4.1 Functional Requirements

Functional requirements describe the core operations that the system must perform.

1. Network Initialization

The system should initialize the SDN environment by setting up hosts, switches, and the SDN controller. This forms the basic network structure required for communication and data transmission.

2. Path Discovery

The system should identify all possible paths between the source and destination nodes. This enables multiple routing options for effective load balancing.

3. Network Monitoring

The system should continuously monitor network conditions such as bandwidth, latency, and packet loss. This ensures real-time awareness of network performance.

4. Parameter Collection

The system should collect network statistics from switches and update the controller with accurate data required for decision-making.

5. Weight Calculation

The system should calculate the weight of each available path using the weighted formula based on bandwidth, latency, and packet loss.

6. Path Comparison

The system should compare the calculated weights of all available paths to evaluate their performance.

7. Optimal Path Selection

The system should select the path with the minimum weight value, ensuring efficient and reliable data transmission.

8. Flow Rule Installation

The system should install appropriate flow rules in the switches through the SDN controller to enforce the selected routing decisions.

9. Packet Forwarding

The system should forward data packets through the selected optimal path based on installed flow rules.

10. Dynamic Path Update

The system should dynamically update path selection based on changing network conditions to maintain optimal performance.

11. Congestion Handling

The system should detect network congestion and switch to alternative paths to avoid performance degradation.

12. Performance Monitoring and Logging

The system should log network parameters and performance metrics for analysis and evaluation.

1.4.2 Non-Functional Requirements

Non-functional requirements define how the system performs its operations.

Performance: The system should provide low latency and high throughput.

Scalability: The system should handle increasing number of hosts and network size.

Reliability: The system should operate continuously without failure.

Efficiency: The system should optimize resource usage and minimize packet loss.

Flexibility: The system should adapt to dynamic network conditions.

1.4.3 Hardware Requirements

Computer/Laptop with minimum 8 GB RAM

Processor: Intel i5 or above

Network interface support

1.4.4 Software Requirements

Operating System: Linux (Ubuntu preferred)

Programming Language: Python

SDN Controller: Ryu / POX

Network Emulator: Mininet

Protocol: OpenFlow

Chapter 2

Literature Review & Existing Digital Solutions

2.1 Summary of Survey

A survey of existing load balancing techniques in Software Defined Networking (SDN) has been conducted to understand different approaches used for traffic distribution and their limitations. Load balancing is an essential function in SDN to ensure efficient utilization of network resources and to improve overall performance.

One of the most basic load balancing techniques is the **Round Robin algorithm**, which distributes incoming traffic equally among all available paths. This method is simple and easy to implement; however, it does not consider the current state of the network. As a result, it may lead to inefficient resource utilization and congestion, especially when network conditions vary.

Another widely used method is the **Least Connection algorithm**, which selects the path with the minimum number of active connections. This approach performs better than Round Robin in some cases, as it considers the load on each path. However, it still fails to account for important network parameters such as bandwidth availability and delay, which can affect overall performance.

To improve performance, some approaches use a **weighted load balancing method based on bandwidth**. In this technique, paths are selected based on their available bandwidth, giving preference to paths with higher capacity. While this method provides better results than traditional techniques, it only considers a single parameter and does not take into account other important factors such as latency and packet loss.

Therefore, existing methods either ignore network conditions or consider only limited parameters. This highlights the need for a more efficient load balancing approach that can utilize multiple parameters for better decision-making in SDN.

2.2 Identified Gaps

From the survey of existing load balancing techniques such as Round Robin, Least Connection, and bandwidth-based weighted methods, several limitations have been identified. These limitations highlight the need for a more efficient and adaptive load balancing approach in Software Defined Networking (SDN).

The **Round Robin algorithm** distributes traffic equally without considering the actual network conditions. This results in inefficient utilization of resources, as some paths may become congested while others remain underutilized.

The **Least Connection method** improves traffic distribution by selecting the path with the minimum number of active connections. However, it does not consider important performance parameters such as bandwidth and latency, which may lead to suboptimal routing decisions.

Similarly, the **weighted load balancing approach based only on bandwidth** considers a single parameter while selecting the path. Although it performs better than traditional methods, it ignores other critical factors such as latency and packet loss. This limitation reduces its effectiveness in dynamic network environments where multiple parameters influence performance.

Another major gap in existing methods is the lack of **real-time adaptability**. Many approaches do not dynamically update their decisions based on changing network conditions, which can lead to performance degradation.

Furthermore, most existing techniques do not provide a **comprehensive multi-parameter optimization strategy**, resulting in poor Quality of Service (QoS) under varying traffic loads.

To address these gaps, there is a need for a load balancing mechanism that considers multiple parameters simultaneously, such as bandwidth, latency, and packet loss, and dynamically adapts to network conditions. The proposed weighted load balancing approach in this project aims to overcome these limitations and improve overall network performance.

Chapter 3

System Design

3.1 High-level Architecture

The proposed system for load balancing in Software Defined Networking (SDN) is based on a centralized control architecture, where the control plane is separated from the data plane. This separation enables efficient network management, dynamic decision-making, and flexible traffic control.

The architecture consists of three main components: Application Layer, Control Layer, and Infrastructure Layer.

The Application Layer contains the load balancing application, where the weighted algorithm is implemented. This layer is responsible for defining network policies and decision-making logic based on parameters such as bandwidth, latency, and packet loss.

The Control Layer is managed by the SDN controller, such as the Ryu controller. It acts as the central intelligence of the network. The controller collects network statistics from switches, calculates path weights using the weighted formula, and selects the optimal path for data transmission. It also installs flow rules in the switches using the OpenFlow protocol.

The Infrastructure Layer consists of network devices such as switches and hosts. The switches are responsible for forwarding packets based on the flow rules provided by the controller. Hosts act as the source and destination of data traffic.

The working of the system begins when a host sends a packet. The switch forwards the packet to the controller if no rule is available. The controller monitors network parameters and calculates the weight of each available path using the formula:

$$W=(1/B)+L+PL$$

Based on the calculated weights, the controller selects the optimal path with the

minimum weight and installs the corresponding flow rules in the switches. The packets are then forwarded through the selected path. This process is continuously updated to adapt to changing network conditions.

3.2 Module Description

The proposed system for load balancing in Software Defined Networking (SDN) using a weighted algorithm is divided into several functional modules. Each module performs a specific task to ensure efficient traffic distribution and optimal network performance.

1. Network Monitoring Module

The network monitoring module is responsible for collecting real-time information about the network. It continuously observes important parameters such as bandwidth, latency, and packet loss from different paths in the network. These parameters are essential for evaluating the condition of each path. The collected data is forwarded to the controller for further processing and decision-making.

2. Path Discovery Module

The path discovery module identifies all possible paths between the source and destination nodes. It maintains a list of available paths and ensures that multiple routing options are considered for load balancing. This module plays a crucial role in enabling dynamic path selection in the network.

3. Weight Calculation Module

The weight calculation module computes the weight of each available path based on the collected network parameters. It uses a weighted formula that considers bandwidth, latency, and packet loss. Paths with higher bandwidth and lower latency and packet loss are assigned higher weights. This module ensures that the best possible path is selected for data transmission.

4. Decision Making Module

The decision-making module selects the optimal path for data transmission based on the calculated weights. It compares all available paths and chooses the one with the highest weight. This module ensures efficient load balancing by dynamically adapting to changing network conditions.

5. Flow Rule Installation Module

This module is responsible for installing the selected routing decisions into the network switches. The SDN controller sends flow rules to the switches, instructing them on how to forward packets through the chosen path. This ensures that traffic is routed efficiently without manual intervention.

6. Packet Forwarding Module

. It forwards incoming packets according to the flow rules provided by the controller. This module ensures smooth and efficient transmission of data across the network.

3.3 UML Diagrams

3.3.1 Use Case Diagram

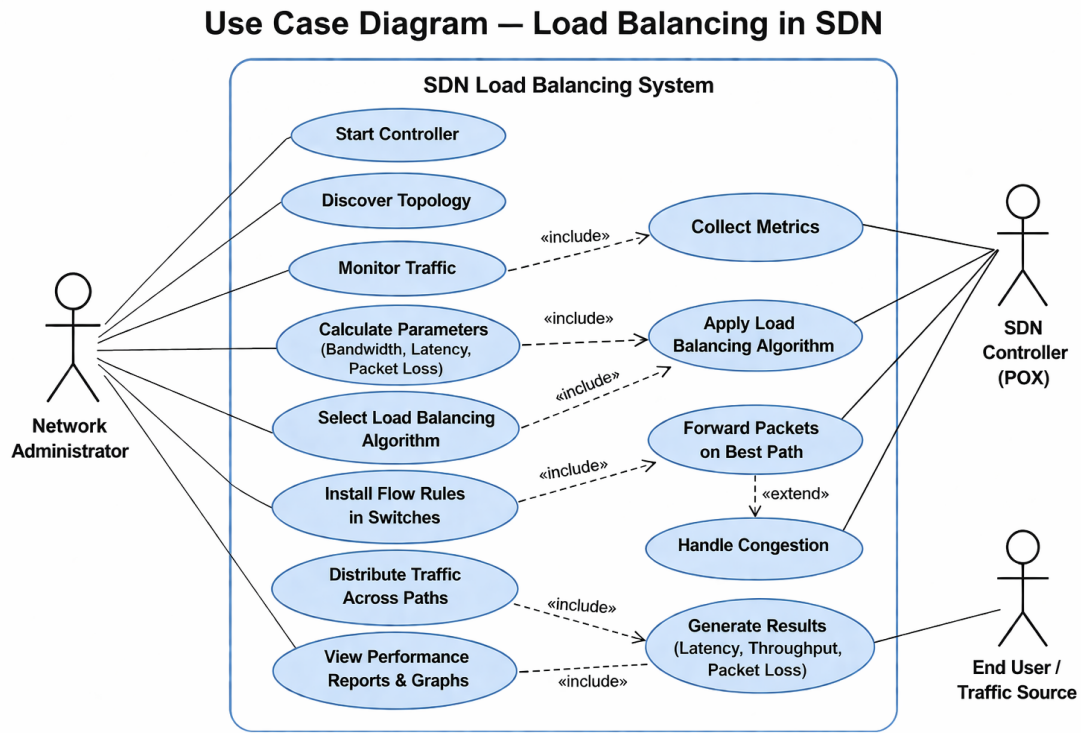


Figure 3.1: Use Case Diagram

3.3.2 Class Diagram

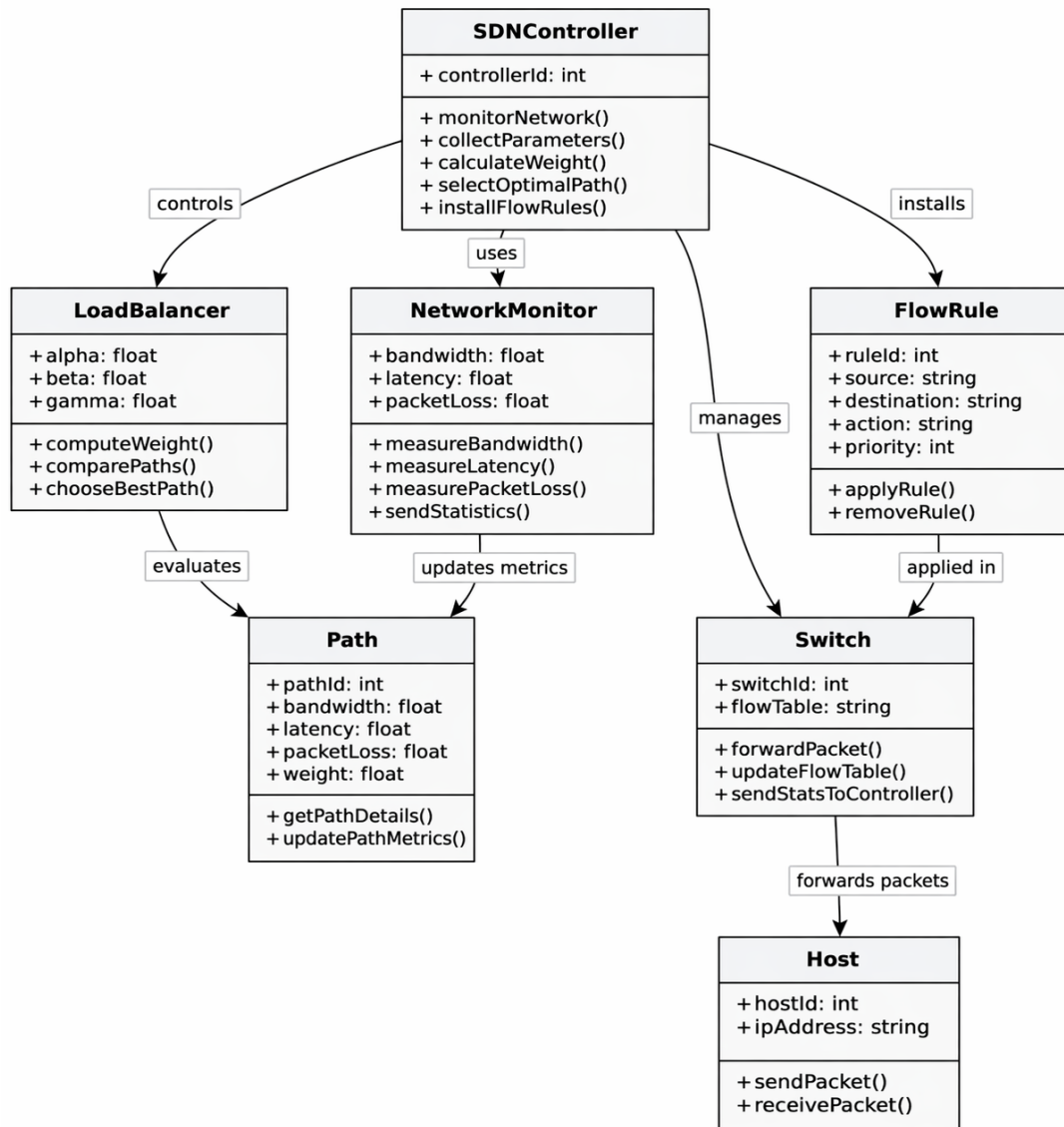


Figure 3.2: Class Diagram

3.3.3 Activity Diagram

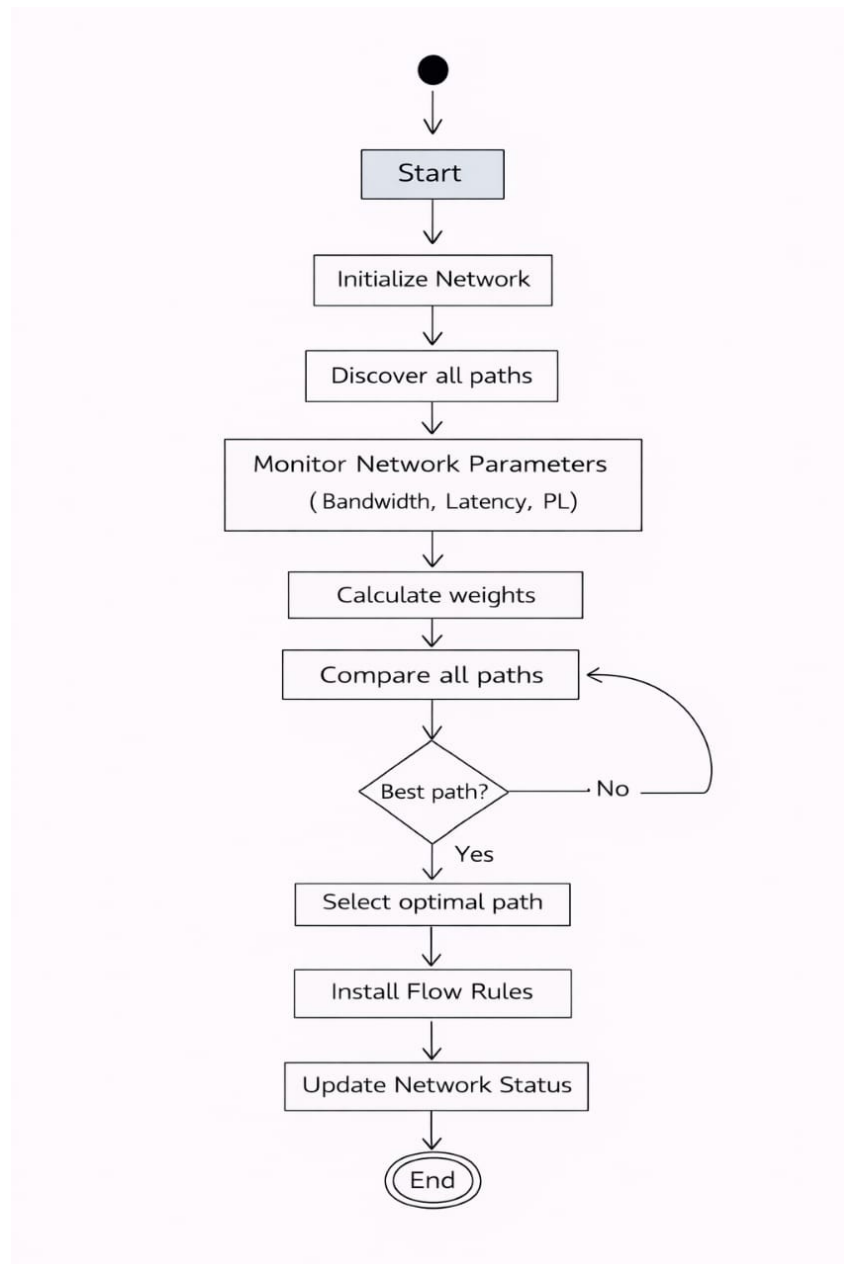


Figure 3.3: Activity Diagram

3.3.4 Sequence Diagram

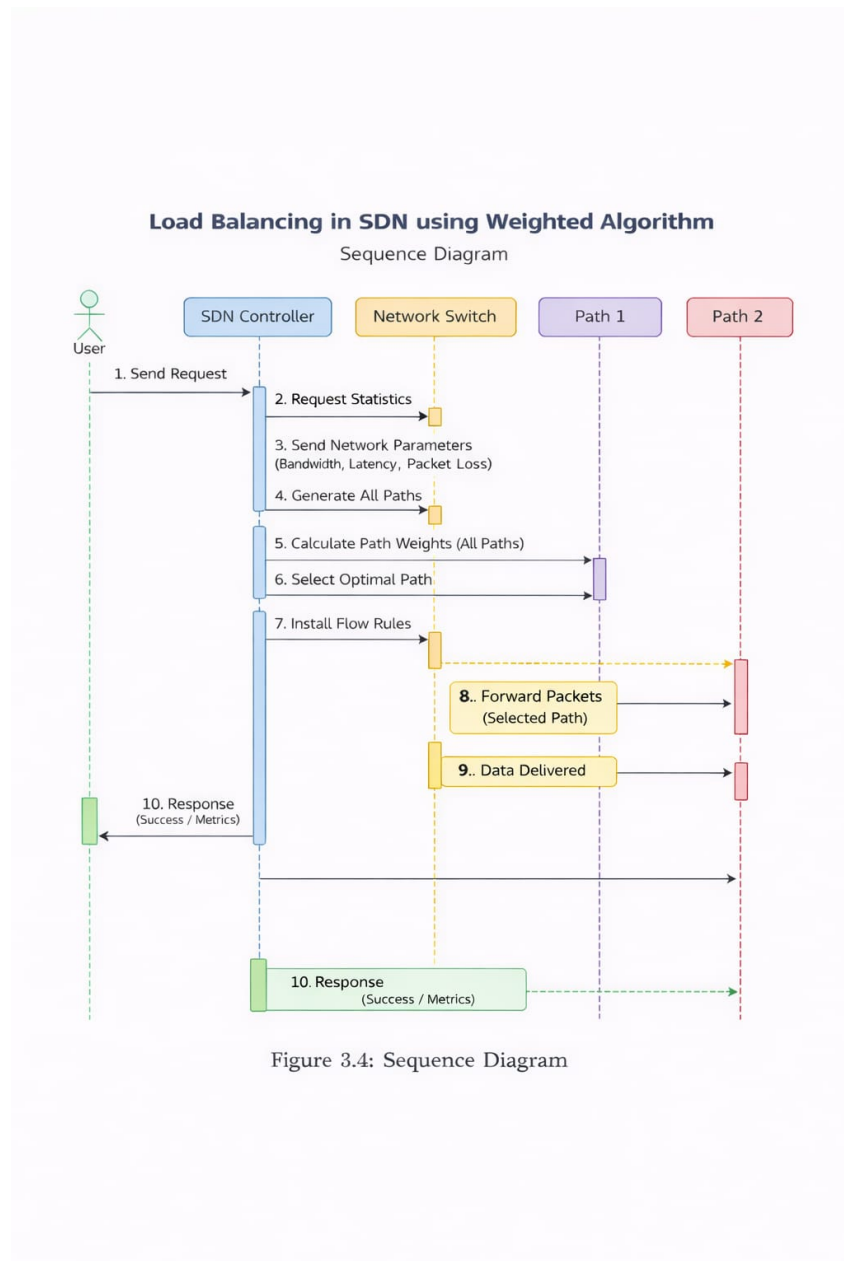


Figure 3.4: Sequence Diagram

3.4 Algorithms/Methods Used

The proposed system uses a weighted load balancing algorithm in Software Defined Networking (SDN) to select the most suitable path for data transmission. The method is based on three important network performance parameters, namely bandwidth, latency, and packet loss. These parameters are used together to evaluate the quality of each available path and to make an efficient routing decision.

In the proposed approach, the SDN controller continuously monitors all possible paths between the source and destination. For each path, the controller collects the values of available bandwidth, latency, and packet loss. Since these parameters have different impacts on network performance, a weighted method is used to combine them into a single score.

In this method, bandwidth is inversely proportional, while latency and packet loss are directly proportional to the weight value. This means that a path with higher bandwidth will result in a lower value of $1/B$, thereby reducing the overall weight, while higher latency and packet loss increase the weight. Hence, the algorithm selects the path with the minimum weight, which represents the most efficient path.

The weighted score of a path is calculated as:

$$W = w_1 \times 1/B + w_2 \times L + w_3 \times PL$$

where,

W = Weight of the path

B = Available bandwidth

L = Latency

PL = Packet loss

$w_1 + w_2 + w_3$ = Weight factors assigned to each parameter

The values of w_1, w_2 and w_3 are selected such that :

$$w_1 + w_2 + w_3 = 1$$

This ensures balanced contribution of all parameters in the decision-making process. After calculating the weights of all available paths, the controller selects the path with the minimum weight value and installs the corresponding flow rules in the switches. Thus, the proposed method provides an efficient and adaptive load balancing mechanism by considering multiple network parameters simultaneously, leading to improved network performance, reduced delay, and minimized packet loss.

Steps of the Algorithm:

Initialize the SDN network and identify all available paths.

Monitor network parameters for each path.

Measure bandwidth, latency, and packet loss.

Calculate the weight of each path using the formula.

Compare the weights of all paths.

Select the path with the minimum weight.

Install flow rules in switches through the controller.

Forward packets through the selected path.

Repeat the process dynamically as network conditions change.

Method Used

The method used in this project is a multi-parameter weighted load balancing method that considers bandwidth, latency, and packet loss simultaneously. This approach provides better performance compared to traditional methods by adapting to real-time network conditions.

Chapter 4

Implementation

4.1 Technology Stack

The implementation of the proposed load balancing system in Software Defined Networking (SDN) requires a combination of software tools, programming languages, and networking frameworks. The selected technology stack ensures efficient simulation, implementation, and testing of the weighted load balancing algorithm.

1. Programming Language

Python is used as the primary programming language for implementing the load balancing algorithm. It provides simplicity, flexibility, and strong support for networking and SDN-based applications. Python is also widely supported by SDN controllers such as Ryu and POX.

2. SDN Controller

The Ryu Controller (or POX Controller) is used as the central control unit of the network. It is responsible for:

- Managing network devices

- Monitoring network conditions

- Implementing the weighted load balancing algorithm

- Installing flow rules in switches

Ryu is preferred due to its simplicity, modular design, and strong support for OpenFlow protocol.

3. Network Emulator

Mininet is used as the network emulator to create a virtual SDN environment. It allows the creation of hosts, switches, and links on a single system. Mininet helps in:

- Simulating real network topologies

- Testing load balancing algorithms

- Measuring performance parameters such as latency and packet loss

4. Protocol Used

The OpenFlow protocol is used for communication between the SDN controller and the switches. It enables the controller to:

Send flow rules to switches

Control packet forwarding decisions

Collect network statistics

5. Operating System

A Linux-based operating system (Ubuntu) is used for implementation. It provides:

Better support for SDN tools Compatibility with Mininet and controllers Stable and efficient performance

6. Development Tools Text editors such as VS Code or Nano for writing code Terminal/Command Line for executing programs Python libraries for networking and simulation

4.2 Key Features Implemented

The proposed load balancing system in Software Defined Networking (SDN) incorporates several important features to ensure efficient traffic management and improved network performance. These features are designed based on the weighted algorithm that considers bandwidth, latency, and packet loss.

1. Dynamic Load Balancing

The system dynamically distributes network traffic across multiple available paths. The path selection is continuously updated based on real-time network conditions, ensuring efficient utilization of resources and avoiding congestion.

2. Multi-Parameter Decision Making

Unlike traditional methods, the system considers multiple parameters for decision making, including bandwidth, latency, and packet loss. This ensures a more accurate evaluation of path performance and leads to better routing decisions.

3. Real-Time Network Monitoring

The system continuously monitors network parameters such as bandwidth, latency, and packet loss. This real-time monitoring helps the controller to adapt to changing network conditions and make informed decisions.

4. Weighted Path Selection

A weighted algorithm is used to calculate the cost of each available path. Based on the formula involving $1/B$, latency, and packet loss, the system selects the path with the minimum weight, ensuring optimal performance.

5. Centralized Control using SDN

The SDN controller manages the entire network from a central point. This centralized control simplifies network management, improves scalability, and enables efficient implementation of the load balancing algorithm.

6. Efficient Resource Utilization

The system ensures proper utilization of network resources by avoiding overloaded paths and distributing traffic evenly. This leads to improved throughput and reduced network congestion.

7. Reduced Latency and Packet Loss

By selecting optimal paths, the system minimizes transmission delay and packet loss, thereby improving the quality of service (QoS) in the network.

8. Scalable and Flexible Design

The proposed system can easily be extended to larger network topologies. It is flexible enough to adapt to different network conditions and requirements.

Chapter 5

Testing and Validation

5.1 Test Cases

Table 5.1: Test Cases for Load Balancing System

Test ID	Description	Expected Output	Result
T1	Check detection of multiple available paths between source and destination	System should identify all possible paths correctly	Pass
T2	Monitor bandwidth of available paths	System should collect bandwidth values successfully	Pass
T3	Monitor latency of available paths	System should measure latency correctly	Pass
T4	Monitor packet loss of available paths	System should detect packet loss accurately	Pass
T5	Calculate path weight using bandwidth, latency, and packet loss	System should compute weight for each path correctly	Pass
T6	Select path with minimum calculated weight	System should choose the optimal path correctly	Pass

5.2 Performance Analysis

The performance of the proposed load balancing system in Software Defined Networking (SDN) is evaluated based on key network parameters such as throughput, latency, packet loss, and bandwidth utilization. These parameters are used to measure the efficiency and effectiveness of the weighted load balancing algorithm.

1. Throughput Analysis

Throughput refers to the amount of data successfully transmitted over the network in a given time. The proposed system achieves higher throughput because it selects paths with better bandwidth availability and lower congestion. By avoiding overloaded paths, the system ensures efficient data transmission.

2. Latency Analysis

Latency is the time delay experienced during data transmission. The weighted algorithm minimizes latency by selecting paths with lower delay values. As a result, the system provides faster data delivery compared to traditional load balancing techniques.

3. Packet Loss Analysis

Packet loss occurs when data packets fail to reach their destination. The proposed system reduces packet loss by avoiding paths with poor network conditions. This improves reliability and ensures better quality of service.

4. Bandwidth Utilization

Bandwidth utilization measures how effectively the available network capacity is used. The system distributes traffic across multiple paths based on their capacity, leading to optimal utilization of bandwidth and preventing congestion.

5. Comparison with Traditional Methods

When compared with traditional load balancing techniques such as Round Robin and Least Connection, the proposed weighted algorithm shows significant improvements in all performance metrics. Traditional methods do not consider real-time network parameters, which leads to inefficient routing decisions.

5.3 User Feedback

User feedback is an important aspect of evaluating the effectiveness and usability of the proposed load balancing system. The system was tested in a simulated SDN environment, and feedback was collected based on its performance, reliability, and ease of implementation.

The users observed that the system provides efficient traffic distribution across multiple paths, resulting in improved network performance. The use of a weighted algorithm based on bandwidth, latency, and packet loss was found to be effective in selecting the optimal path for data transmission.

The system demonstrated noticeable improvements in terms of throughput and bandwidth utilization, as users experienced faster and more stable data transmission. The reduction in **latency and packet loss** also contributed to a better overall network experience.

Users also appreciated the **dynamic nature of the system**, as it adapts to changing network conditions in real time. This feature helps in avoiding congestion and ensures continuous smooth operation of the network.

Additionally, the centralized control provided by the SDN controller made the system easier to manage and monitor. The implementation using tools like Mininet and Ryu was found to be user-friendly and suitable for simulation and testing purposes.

However, some users suggested that the system could be further improved by incorporating advanced techniques such as machine learning for predictive load balancing and real-time optimization in large-scale networks.

Chapter 6

Results and Discussion

The proposed load balancing system in Software Defined Networking (SDN) using a weighted algorithm was implemented and tested in a simulated environment. The performance of the system was evaluated based on key parameters such as throughput, latency, packet loss, and bandwidth utilization. The results obtained from the experiments demonstrate the effectiveness of the proposed approach in improving overall network performance.

The experimental results show that the proposed weighted algorithm performs significantly better than traditional load balancing methods such as Round Robin and Least Connection. By considering multiple parameters—bandwidth, latency, and packet loss—the system is able to select the most efficient path for data transmission.

In terms of throughput, the proposed method achieves higher data transmission rates because it avoids congested paths and utilizes available bandwidth effectively. This leads to better utilization of network resources and improved data delivery.

For latency, the system shows reduced delay compared to traditional methods. Since the algorithm prioritizes paths with lower latency, data packets are delivered faster, resulting in improved responsiveness of the network.

The analysis of packet loss indicates that the proposed system minimizes data loss during transmission. By avoiding paths with higher packet loss, the system ensures reliable communication and maintains data integrity.

The bandwidth utilization is also improved, as the algorithm distributes traffic based on the capacity of each path. This prevents overloading of certain paths while ensuring that all available resources are used efficiently. The discussion of results clearly indicates that traditional load balancing techniques fail to adapt to dynamic network conditions because they do not consider multiple parameters. In contrast, the proposed weighted algorithm provides a more intelligent and adaptive solution by combining bandwidth, latency, and packet loss into the decision-making process.

Chapter 7

Deployment

7.1 Build Process

The build process of the proposed load balancing system in Software Defined Networking (SDN) involves the setup, development, and execution of the system using the selected technology stack. The process ensures proper implementation of the weighted load balancing algorithm and its integration with the SDN environment.

Initially, the required software tools such as Mininet, Ryu controller, and Python were installed and configured on a Linux-based system (Ubuntu). Mininet was used to create a virtual network topology consisting of hosts, switches, and links, while the Ryu controller was used to manage the network and implement the control logic.

After setting up the environment, a custom network topology was designed in Mininet with multiple paths between source and destination nodes. This allowed the system to test different routing options for load balancing. The controller application was then developed using Python to implement the weighted load balancing algorithm.

The controller continuously monitored network parameters such as bandwidth, latency, and packet loss from different paths. Based on these parameters, the algorithm calculated the weight of each path using the defined formula and selected the optimal path for data transmission.

Once the best path was selected, the controller installed flow rules in the OpenFlow switches to forward packets accordingly. The system was then executed and tested under different network conditions to evaluate its performance.

Finally, the results were collected and analyzed using performance metrics such as throughput, latency, packet loss, and bandwidth utilization. The build process ensured that all components worked together effectively to achieve dynamic and efficient load balancing in the SDN environment.

7.2 Installation Steps

The installation of the proposed load balancing system in Software Defined Networking (SDN) involves setting up the required software tools and configuring the environment for simulation and implementation. The following steps describe the installation process:

Step 1: Install Operating System

Install a Linux-based operating system such as Ubuntu (recommended version: 20.04 or later). Ensure that the system has proper internet connectivity and required system resources.

Step 2: Update System Packages

Open the terminal and update the system packages using the following commands:

```
sudo apt update
```

```
sudo apt upgrade
```

Step 3: Install Mininet

Install Mininet to create and simulate the SDN network:

```
sudo apt install mininet
```

Verify installation:

```
sudo mn -test pingall
```

Step 4: Install Ryu Controller

Install Ryu controller using pip:

```
pip install ryu
```

verify installation:

```
ryu-manager -version
```

Step 5: Install Python and Required Libraries

Ensure Python is installed (usually pre-installed in Ubuntu):

```
python3 --version
```

install required libraries:

```
pip install networkx matplotlib
```

Step 6: Setup Project Files

Create a project directory and place your controller code (Python file) inside it:

```
mkdir sdn load balancing
```

```
cd sdn loadbalancing
```

Step 7: Run Ryu Controller

Start the controller with your application file:

```
ryu-manager your controller file.py
```

Step 8: Run Mininet Topology

Open a new terminal and start Mininet with a custom topology:

```
sudo mn -topo linear,3 -controller remote
```

Step 9: Test Network Connectivity

In the Mininet CLI, test connectivity between hosts:

pingall

Step 10: Execute Load Balancing Generate traffic and observe path selection and load balancing behavior. Monitor outputs in the controller terminal.

7.3 Deployment Platform

The proposed system is deployed using the following platform and tools:

Operating System:

Ubuntu Linux (preferred for SDN implementation and compatibility with tools)

Network Emulator:

Mininet is used to create a virtual SDN network consisting of hosts, switches, and links

SDN Controller:

Ryu Controller is used for centralized network control and implementation of the load balancing algorithm

Programming Language:

Python is used for developing the controller logic and implementing the weighted algorithm

Communication Protocol:

OpenFlow protocol is used for communication between the controller and switches

Simulation Environment:

Virtual network environment created using Mininet for testing and analysis

Hardware Platform:

Standard computer/laptop with minimum 8 GB RAM and Intel i5 processor or higher

Execution Platform:

Command-line interface (Terminal) used for running Mininet and controller programs

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The project titled “Load Balancing in Software Defined Networking using Weighted Algorithm based on Bandwidth, Latency, and Packet Loss” has been successfully designed and implemented. The main objective of the project was to develop an efficient load balancing mechanism that improves network performance by considering multiple parameters for path selection.

The proposed system utilizes the centralized control feature of Software Defined Networking (SDN) to monitor network conditions and dynamically select the optimal path for data transmission. By incorporating bandwidth, latency, and packet loss into a weighted algorithm, the system ensures more accurate and efficient routing decisions compared to traditional load balancing techniques.

The experimental results demonstrate that the proposed approach significantly improves network performance in terms of throughput, latency, packet loss, and bandwidth utilization. The system effectively distributes traffic across multiple paths, reduces congestion, and enhances overall Quality of Service (QoS).

Furthermore, the use of tools such as Mininet and Ryu controller provides a flexible and efficient environment for simulation and testing. The implementation confirms that the weighted load balancing method is more adaptive and reliable in handling dynamic network conditions.

In conclusion, the proposed system provides an effective solution for load balancing in SDN by utilizing multiple performance parameters. It offers improved efficiency, better resource utilization, and enhanced network reliability, making it suitable for modern networking environments.

8.2 Future Enhancements

Although the proposed weighted load balancing system in Software Defined Networking (SDN) provides improved performance, there are several areas where the system can be further enhanced to achieve better efficiency and scalability.

In the future, the system can be extended by incorporating **Machine Learning (ML) techniques** to predict network conditions and make intelligent routing decisions. This would allow the system to proactively handle congestion and improve performance in dynamic network environments.

The current system considers three parameters: bandwidth, latency, and packet loss. It can be further enhanced by including additional parameters such as **jitter, link reliability, and energy consumption** to make more comprehensive decisions.

Another possible enhancement is the development of a **graphical user interface (GUI)** for better visualization and monitoring of network performance. This would make the system more user-friendly and easier to manage.

Security features can also be integrated to protect the network from potential threats such as **Distributed Denial of Service (DDoS) attacks**. This would enhance the reliability and robustness of the system.

Additionally, the system can be scaled to support large-scale networks and cloud-based environments, where efficient load balancing is critical for performance.

Bibliography

- [1] B. A. A. Nunes et al., “A survey of software-defined networking: Past, present, and future of programmable networks,” *IEEE Communications Surveys & Tutorials*, vol. 16, no. 3, pp. 1617–1634, 2014.
- [2] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, “Hedera: Dynamic flow scheduling for data center networks,” in *Proc. USENIX NSDI*, 2010, pp. 19–19.
- [3] T. Nadeau and K. Gray, *SDN: Software Defined Networks*, 1st ed. Sebastopol, CA, USA: O’Reilly Media, 2013.
- [4] Ryu SDN Framework Community, “Ryu Controller Documentation,” Available: <https://osrg.github.io/ryu/>
- [5] Mininet Project, “Mininet: An Instant Virtual Network on your Laptop,” Available: <http://mininet.org/>

Appendix A

Appendix

A.1 Source Code (Optional)

A.2 Additional Screenshots